

Fase 5 — Panel Administrativo (tipo CMS)

Distribución de tareas, especificaciones técnicas y convención de ramas para el equipo de desarrollo.

Proyecto: Plataforma Web — Centro de Especialización Ejecutiva (CEE)

Fase actual: 5 de 7 — Panel Administrativo

Equipo asignado: Diana · Tom · Isabel · Renato · Elvis

Repositorio: apps/admin (monorepo Turborepo)

Fecha: 19 de junio de 2026

Objetivo: backoffice para que el personal del CEE gestione contenido (cursos, sílabos, ventas) de forma autónoma, sin tocar código.

Stack de la fase: React + Vite + TS, shadcn/ui, Tailwind, Zustand, recharts, React Router v6, `VITE_USE MOCKS=true`

- ⚠ **Decisión cerrada:** el backoffice real vive en `apps/admin` (app separada del público). No se sigue construyendo en `apps/web` .
- ⚠ **Fuera de alcance:** conexión a backend real (eso corresponde a Fase 6). Todo el trabajo de esta fase opera sobre datos mock.

Elvis

Infraestructura del panel (base para todo el equipo)

Por qué va primero: Diana, Tom, Isabel y Renato montan sus páginas sobre esta base. Si esto no está listo, nadie más puede integrar.

1

Confirmar apps/admin como app independiente

- Verificar que tiene su propio vite.config.ts, package.json y alias @/ funcionando.
- Resolver duplicidad: si quedaron pantallas admin dentro de apps/web, eliminarlas o migrarlas.
- Confirmar que pnpm dev levanta apps/admin en un puerto distinto a apps/web.

2

ProtectedRoute component (apps/admin/src/components/ProtectedRoute.tsx)

- Prop requiredRole="admin".
- Lee sesión desde authStore (mock por ahora: usuario admin hardcodeado o flag en mock).
- Si no hay sesión válida o el rol no coincide → redirect a /login (o pantalla "Acceso denegado").
- Wrapper reutilizable: <ProtectedRoute requiredRole="admin"><DashboardPage/></ProtectedRoute>

3

Layout del admin (apps/admin/src/layouts/AdminLayout.tsx)

- Sidebar fijo con navegación: Dashboard / Cursos / Ventas (iconos + label).
- Header superior: nombre del usuario admin, botón cerrar sesión.
- <Outlet/> para renderizar la página activa.
- Responsive: sidebar colapsable en mobile/tablet (drawer o ícono hamburguesa).

4

Router de apps/admin (apps/admin/src/router.tsx)

- Rutas con lazy() + <Suspense>: / (Dashboard), /cursos (lista), /cursos/nuevo (crear), /cursos/:id/editar (editar), /ventas (panel de ventas).
- Todas las rutas envueltas en ProtectedRoute.

5

Mock de auth admin (apps/admin/src/mocks/auth.ts)

- Usuario admin de prueba con rol "admin".
- Función mockLogin() para simular sesión durante desarrollo.

Done cuando:

pnpm dev levanta apps/admin, login mock funciona, y las 4 rutas existen (aunque sea con placeholder) protegidas por rol. Sin esto, el resto del equipo no puede integrar sus páginas.

Rama:

fase5-infra-admin-protected-route

Diana

Dashboard

1

DashboardPage (apps/admin/src/pages/DashboardPage.tsx)

- Saludo/bienvenida con nombre del usuario admin.
- Fecha actual y resumen rápido del estado del sistema.

2

Tarjetas de KPIs resumidos (components/dashboard/SummaryCard.tsx)

- 4 tarjetas: Cursos Publicados, Cursos en Borrador, Ventas del Mes, Usuarios Registrados.
- Cada tarjeta: ícono, número grande, label, variación vs. mes anterior (texto tipo "+12%", mock).
- Componente reutilizable que recibe {icon, label, value, trend} como props.

3

Accesos directos

- 2 botones/tarjetas grandes: "Gestionar Cursos" → /cursos, "Ver Ventas" → /ventas.

4

Mini-resumen de actividad reciente (opcional si da el tiempo)

- Lista de últimos 5 cursos creados/editados (mock), con fecha y autor.

5

Mock de datos (apps/admin/src/mocks/dashboard.ts)

- Fixture con los números de los KPIs y la actividad reciente.

Done cuando:

Dashboard renderiza con las 4 tarjetas de KPIs + accesos directos, responsive, sin errores de consola. Datos vienen de dashboardService (mock), no hardcodeados directo en el componente.

Rama:

fase5-dashboard

1 CoursesListPage (apps/admin/src/pages/CoursesListPage.tsx)

- Tabla (shadcn Table) con columnas: Nombre, Categoría, Modalidad, Precio, Estado, Fecha creación, Acciones.

2 StatusBadge (components/courses/StatusBadge.tsx)

- Variantes de color: Publicado (verde), Borrador (gris), En Revisión (amarillo/naranja).
- Reutilizable también en el formulario de Isabel y la futura integración real.

3 Acciones por fila

- Botón Editar → navega a /cursos/:id/editar.
- Botón Cambiar estado → dropdown/menu con las 3 opciones (actualiza mock en memoria).
- Botón Eliminar → modal de confirmación (AlertDialog de shadcn) antes de borrar.
- Botón Nuevo curso (arriba de la tabla) → navega a /cursos/nuevo.

4 Búsqueda y filtro

- Input de búsqueda por nombre del curso (filtra en cliente sobre el mock).
- Filtro por estado (select: Todos / Publicado / Borrador / En Revisión).

5 Estado vacío y loading

- Skeleton o spinner mientras "carga" el mock.
- Mensaje cuando no hay resultados de búsqueda.

6 coursesService (apps/admin/src/services/coursesService.ts)

- Funciones mock: getCourses(), updateCourseStatus(id, status), deleteCourse(id).
- Usa fixtures basados en la interfaz Course de @cee/types.

Done cuando: la tabla lista, filtra y busca cursos mock; cambiar estado y eliminar funcionan en memoria (sin persistencia real, pero con feedback visual inmediato — toast de confirmación).

Rama: `fase5-gestion-cursos`

1 CourseFormPage (apps/admin/src/pages/CourseFormPage.tsx)

- Mismo componente para crear (/cursos/nuevo) y editar (/cursos/:id/editar), detecta el modo por la presencia de :id en la ruta.
- Si es edición, precarga los datos del curso desde el mock.

2 Campos del formulario

- Nombre del curso (texto, requerido).
- Descripción (textarea, requerido).
- Precio (número, requerido, validar mayor a 0).
- Categoría (select, requerido).
- Modalidad (select: Presencial / Virtual / Híbrido).
- Moodle Course ID (texto, requerido — campo clave para la integración de Fase 6).
- Estado inicial (Publicado / Borrador / En Revisión).

3 Subida de sílabo (PDF)

- Input file restringido a .pdf.
- Validar tipo de archivo y tamaño máximo (ej. 10MB).
- Mostrar nombre del archivo seleccionado + opción de quitarlo.
- Si es edición y ya existe un sílabo, mostrar el nombre actual con opción de reemplazar.

4 Validación del formulario

- Usar react-hook-form (o el patrón que ya usen en apps/web) + validación de campos requeridos.
- Mensajes de error claros bajo cada campo.
- Deshabilitar botón "Guardar" mientras hay errores o está enviando.

5 Guardar / feedback

- Botón "Guardar curso" → llama a coursesService.createCourse() o updateCourse() (mock).
- Toast de éxito y redirect a /cursos al guardar.
- Botón "Cancelar" → vuelve a la lista sin guardar.

Done cuando: se puede crear un curso nuevo y editar uno existente, con validación funcionando y subida de PDF simulada (sin backend real).

Rama: `fase5-registrar-editar-curso`

Renato

Ventas

1 SalesPage (apps/admin/src/pages/SalesPage.tsx)

2 KPIs principales

- Reutilizar SummaryCard de Diana si ya está listo, o coordinar el componente compartido.
- Total Ventas (cantidad de transacciones).
- Ingresos (monto total, formateado en moneda).
- Tasa de Conversión (% mock, ej. visitas vs. compras).

3 Gráfico de tendencia (recharts)

- Línea o barras: ventas por semana/mes (últimos 6 períodos, mock).
- Tooltip al pasar el mouse mostrando el valor exacto.
- Responsive (se ajusta en mobile).

4 Desglose por curso

- Tabla o gráfico de barras horizontal: top cursos más vendidos con cantidad e ingresos.

5 Filtro de rango de fechas (si da el tiempo)

- Selector simple: Última semana / Último mes / Último trimestre — recalcula KPIs y gráfico sobre el mock.

6 salesService + mocks

- apps/admin/src/mocks/sales.ts y apps/admin/src/services/salesService.ts.
- Fixture con datos de ventas por período y por curso.

Done cuando: KPIs, gráfico de tendencia y desglose por curso renderizan con datos mock, responsive, sin errores de consola.

Rama: fase5-ventas

✓ Done de la Fase 5 (criterio global)

- Las 4 páginas (Dashboard, Cursos, Formulario, Ventas) viven en apps/admin, protegidas por ProtectedRoute requiredRole="admin".
- CRUD de cursos funciona end-to-end sobre mocks (crear, listar, editar, cambiar estado, eliminar).
- Ventas muestra KPIs + gráfico recharts + desglose por curso.
- Todo responsive (mobile / tablet / desktop), sin errores de consola.
- Cada quien hizo merge de su rama a develop (o la rama base que usen) sin romper el build de los demás.

Renombrado de ramas

Cada quien debe renombrar su rama de trabajo para que incluya **fase + tarea**, así queda trazable en el historial de Git.

```
git branch -m <rama-anterior> fase5-<nombre-tarea>

git push origin :<rama-anterior>          # borra la vieja en remoto
git push origin -u fase5-<nombre-tarea>    # sube la nueva con el nombre correcto
```

Persona	Tarea	Rama nueva
Elvis	Infraestructura del panel	fase5-infra-admin-protected-route
Diana	Dashboard	fase5-dashboard
Tom	Gestión de Cursos	fase5-gestion-cursos
Isabel	Registrar / Editar curso	fase5-registrar-editar-curso
Renato	Ventas	fase5-ventas